

# Gaussian-LIC: Real-Time Photo-Realistic SLAM with Gaussian Splatting and LiDAR-Inertial-Camera Fusion

Xiaolei Lang<sup>1,†</sup>, Laijian Li<sup>1,†</sup>, Chenming Wu<sup>2</sup>, Chen Zhao<sup>2</sup>, Lina Liu<sup>1</sup>, Yong Liu<sup>1</sup>, Jiajun Lv<sup>1,\*</sup>, Xingxing Zuo<sup>3,\*</sup>

**Abstract**—In this paper, we present a real-time photo-realistic SLAM method based on marrying Gaussian Splatting with LiDAR-Inertial-Camera SLAM. Most existing radiance-field-based SLAM systems mainly focus on bounded indoor environments, equipped with RGB-D or RGB sensors. However, they are prone to decline when expanding to unbounded scenes or encountering adverse conditions, such as violent motions and changing illumination. In contrast, oriented to general scenarios, our approach additionally tightly fuses LiDAR, IMU, and camera for robust pose estimation and photo-realistic online mapping. To compensate for regions unobserved by the LiDAR, we propose to integrate both the triangulated visual points from images and LiDAR points for initializing 3D Gaussians. In addition, the modeling of the sky and varying camera exposure have been realized for high-quality rendering. Notably, we implement our system purely with C++ and CUDA, and meticulously design a series of strategies to accelerate the online optimization of the Gaussian-based scene representation. Extensive experiments demonstrate that our method outperforms its counterparts while maintaining real-time capability. Impressively, regarding photo-realistic mapping, our method with our estimated poses even surpasses all the compared approaches that utilize privileged ground-truth poses for mapping. Our code will be released on project page [https://xingxingzuo.github.io/gaussian\\_lic](https://xingxingzuo.github.io/gaussian_lic).

## I. INTRODUCTION

Simultaneous localization and mapping (SLAM) is a pivotal technique in mixed reality and robotics. In recent years, emerging radiance field technologies such as Neural Radiance Fields (NeRF) [1] and 3D Gaussian Splatting (3DGS) [2] have carved out a new direction in the realm of SLAM, namely radiance-field-based SLAM [3], aiming to simultaneously localize accurately and construct photo-realistic 3D maps. This holds significant importance for enhancing the capabilities of robots to model, understand, and interact with natural environments in real time [4–7].

Compared to NeRF which involves computationally intensive volumetric rendering based on ray tracing, 3DGS offers fast rendering speed with superior visual quality, showcasing a greater potential in real-time SLAM applications. Yet there still remain several issues for 3DGS-based SLAM. First, similar to existing NeRF-based SLAM [8–16], most 3DGS-based SLAM [17–29] focus mainly on indoor scenarios using RGB-D or RGB sensors. When it comes to unbounded outdoor scenes or encountering challenging conditions such

as violent ego-motion, varying illumination, and lack of visual textures, the performance of existing 3DGS-based SLAM might dramatically diminish. Second, for the outdoor scenarios, the modeling of the sky and varying camera exposure becomes important. Ignoring them can incur significant artifacts to 3DGS in unbounded scenes. Third, real-time performance is still a bottleneck. Constraining the number of or the dimension of attributes of 3D Gaussian representation [18, 22] could be effective indoors but might lead to deteriorated performance in unbounded outdoor scenarios.

To address the problems above, we propose Gaussian-LIC, a photo-realistic SLAM system that tightly fuses LiDAR-Inertial-Camera data for robust pose tracking and photo-realistic reconstruction with 3D Gaussian map representation **all in real time**. The 3D Gaussians can be initialized using both the LiDAR points and visual SFM (struct-from-motion) points (obtained by online triangulation). Both the sky and changing camera exposure are taken into consideration in our system for higher-quality reconstruction. Importantly, we carefully design acceleration strategies for Gaussian optimization in CUDA implementation. In summary, our main contributions are as follows:

- We present the **first** real-time photo-realistic SLAM with Gaussian Splatting and LiDAR-Inertial-Camera fusion in unbounded outdoor scenarios. Our SLAM framework delivers robust pose estimation while incrementally constructing a high-fidelity Gaussian map, all without the need for extensive post-processing after the sensory data is received.
- For high-quality rendering in broad 3D space, we seamlessly incorporate LiDAR points and 3D visual landmarks obtained from triangulation for initialization of 3D Gaussians. Also, we model the sky and varying camera exposure to better tackle the complex unbounded scenes outdoors.
- We implement our system purely in C++ and CUDA and meticulously design a series of CUDA-related acceleration strategies to boost the training of Gaussians, which will be open-sourced to benefit the community.
- We conduct comprehensive experiments on real-world indoor and outdoor datasets using various types of LiDAR, demonstrating that our method achieves state-of-the-art performance in real-time photorealistic mapping, even under adverse conditions. Remarkably, our approach, using our estimated poses, even outperforms

<sup>1</sup>Institute of Cyber-Systems and Control, Zhejiang University, China.

<sup>2</sup>Baidu VIS, China.

<sup>3</sup>California Institute of Technology, USA.

<sup>†</sup>Contributed equally. \*Corresponding authors.

all compared methods that rely on privileged ground-truth poses for mapping.

## II. RELATED WORK

### A. Photo-Realistic RGB-D or RGB SLAM

Given sequential RGB-D or RGB inputs, abundant works attain brilliant results of localization and photo-realistic mapping in indoor environments. iMAP [8], the first NeRF-based SLAM that employs the implicit neural representation to achieve watertight online reconstruction, has pioneered a new era in SLAM. As a follow-up, NICE-SLAM [9] combines MLPs with hierarchical feature grids, markedly improving the quality of scene representation and realizing outstanding performance in larger indoor rooms. Further, Vox-Fusion [10] utilizes octree to dynamically expand the volumetric neural implicit map, eliminating the need for pre-allocated grids. Adopting hash-grids, tri-planes, and neural point clouds as implicit neural representations respectively, Co-SLAM [11], ESLAM [12], and Point-SLAM [13] get enhancement in both localization and reconstruction. Meanwhile, HERO-SLAM [14] improves the robustness in the optimization of the neural map. NeRF employs time-consuming ray-based volume rendering, while 3DGS utilizes fast point-based rasterization, accelerating image view synthesis and producing promising rendering quality. Notably, GS-SLAM [17], SplatAM [18], and Gaussian-SLAM [19] detailedly elucidate the significant advantages of 3DGS over existing map representations in SLAM tasks for on-line photo-realistic mapping. SGS-SLAM [27], SemGauss-SLAM [28], and NEDS-SLAM [29] integrate semantic feature embedding into Gaussians and demonstrate superb capabilities in online dense semantic mapping. Fusing Generalized Iterative Closest Point (G-ICP) and 3DGS, GS-ICP-SLAM [20] shares covariances of Gaussians between tracking and mapping to minimize redundant computations. CG-SLAM [21] additionally incorporates a depth uncertainty model to select valuable Gaussians and thereby improves tracking performance. By forcing each Gaussian to be either opaque or nearly transparent, RTG-SLAM [22] achieves real-time performance indoors with compact scene representation.

Several works have tried to operate solely on monocular images. Among them, NeRF-SLAM [15] and IG-SLAM [23] utilize the dense depth maps estimated from the tracking front-end DROID-SLAM [30] as additional information to supervise the training of Instant-NGP [31] and 3DGS. Correspondingly, NeRF-VO [16] and MGS-SLAM [24] instead employ the sparse visual odometry DPVO [32] as a faster front-end with network-predicted dense depth for supervision. Concurrently, Photo-SLAM [25] utilizes the classical visual odometry ORB-SLAM3 [33] for accurate pose estimation and reconstructs a hybrid Gaussian map with ORB features. MonoGS [26] introduces geometric regularization to address ambiguities in incremental reconstruction.

### B. Photo-Realistic Multimodal SLAM

RGB-D and RGB approaches can realize fancy results in well-lit indoor scenes, but they would find it difficult to scale

up to complex unbounded outdoor environments or cope with violent motions, lighting changes, and texture missing. Also, RGB-D sensors are often unavailable outdoors. To this end, fusing multimodal sensors such as LiDAR and IMU will be necessary and effective, which has been extensively proven in classical SLAM methods [34–40] characterized by robustness and accuracy. Nevertheless, their primary focus lies in geometric mapping, resulting in geometrically precise yet visually simple representations of the scene. URF [41], DrivingGaussian [42], StreetGaussians [43], PVG [44], LIV-GaussMap [45], and LetsGo [46] perform enhanced photo-realistic reconstruction based on LiDAR-camera fusion, but all of them are offline methods taking a batch input.

Oriented to SLAM tasks, MM-Gaussian [47] leverages accurate geometric structure information from a solid-state LiDAR to substitute the depth of the RGB-D, thus scaling up to the outdoor scenes. Specifically, it first performs a pure LiDAR odometry [48] for pose estimations and further refines the pose via color and depth rendering loss. Similar to SplatAM [18], it optimizes isotropic Gaussians without view-dependent effect initialized from LiDAR points. MM3DGS-SLAM [49] fuses RGB-D camera and IMU measurements in a loosely-coupled manner, to enable more accurate and scale-aware pose estimation. Both of the aforementioned multimodal SLAM approaches are capable of functioning in outdoor environments. However, their mapping performance is suboptimal due to the lack of sky modeling and the challenges posed by varying camera exposure in outdoor settings. Additionally, they are not specially optimized for real-time applications.

## III. METHODOLOGY

Our proposed Gaussian-LIC consists of two main modules: LiDAR-Inertial-Camera odometry with mapping data preparation (Sec. III-B) and 3DGS-based photo-realistic on-line mapping (Sec. III-C) powered by the carefully designed acceleration strategies (Sec. III-D) for real-time performance. Fig. 1 shows an overview of our system.

### A. Preliminary: 3D Gaussian Splatting

To reconstruct a richly detailed dense map for fast and high-quality rendering, we represent the scene as a collection of anisotropic 3D Gaussians [2], each of which contains position  $\mu \in \mathbb{R}^3$  in the world coordinate, scale  $\mathbf{S} \in \mathbb{R}^3$ , rotation  $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ , opacity  $o \in \mathbb{R}$ , and three-degree spherical harmonics  $\mathbf{SH} \in \mathbb{R}^{48}$  capturing the view-dependent appearance of the scene. Representing the Gaussian's ellipsoidal shape, the covariance of each Gaussian is parameterized as:

$$\Sigma = \mathbf{R} \mathbf{S} \mathbf{S}^T \mathbf{R}^T. \quad (1)$$

Given the camera pose  ${}^C_W \mathbf{T} = \{{}^C_W \mathbf{R}, {}^C_W \mathbf{p}_W\}$  which transforms the point  ${}^W \mathbf{p}$  in the world frame  $\{W\}$  into the camera frame  $\{C\}$ , we can splat a 3D Gaussian  $\mathcal{N}(\mu, \Sigma)$  onto the image screen and get a 2D Gaussian  $\mathcal{N}(\mu', \Sigma')$ :

$$\mu' = \pi_c \left( \frac{\hat{\mu}}{\mathbf{e}_3^T \hat{\mu}} \right), \quad \hat{\mu} = {}^C_W \mathbf{R} \mu + {}^C_W \mathbf{p}_W, \quad (2)$$

$$\Sigma' = [\mathbf{J} {}^C_W \mathbf{R} \Sigma {}^C_W \mathbf{R}^T \mathbf{J}^T]_{2 \times 2}, \quad (3)$$

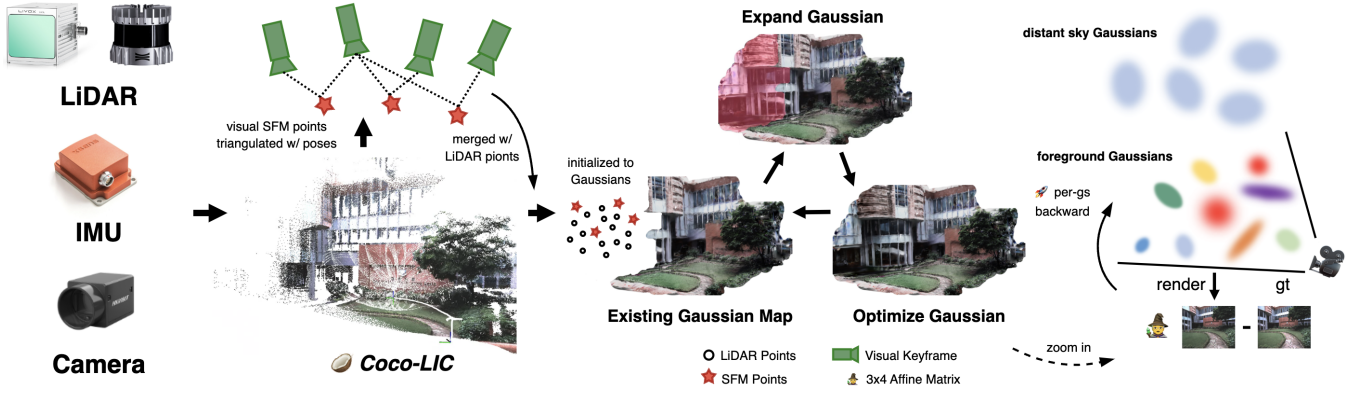


Fig. 1: The pipeline of our novel photo-realistic LiDAR-Inertial-Camera SLAM using 3D Gaussian Splatting.

where  $\mathbf{e}_i$  is a  $3 \times 1$  vector with its  $i$ -th element to be 1 and the other elements to be 0. Then  $\mathbf{e}_3^\top \hat{\boldsymbol{\mu}}$  is the depth  $d$  of the Gaussian in the camera frame.  $\pi_c(\cdot)$  denotes the projection operation which transforms a point on the normalized image plane to a pixel.  $\mathbf{J}$  is the Jacobian of the affine approximation of the projective transformation and  $[\cdot]_{2 \times 2}$  skips the third row and column to acquire a  $2 \times 2$  matrix. The projected 2D Gaussian affects the pixel  $\boldsymbol{\rho} = [u \ v]^\top$  by the weight:

$$\alpha = o \exp \left( -\frac{1}{2} (\boldsymbol{\mu}' - \boldsymbol{\rho})^\top (\boldsymbol{\Sigma}')^{-1} (\boldsymbol{\mu}' - \boldsymbol{\rho}) \right). \quad (4)$$

Sorting the Gaussians in depth order, the color of a pixel  $\boldsymbol{\rho}$  can be efficiently rendered via front-to-back  $\alpha$ -blending:

$$\mathbf{C}(\boldsymbol{\rho}) = \sum_{i=1}^n \mathbf{c}_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad (5)$$

where  $\mathbf{c}$  denotes the RGB color of the Gaussian computed from **SH**. Similarly, we also render the depth map and opacity map by:

$$\mathbf{D}(\boldsymbol{\rho}) = \sum_{i=1}^n d_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad (6)$$

$$\mathbf{O}(\boldsymbol{\rho}) = \sum_{i=1}^n \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j). \quad (7)$$

### B. Multimodal Odometry and Mapping Data Preparation

Given data from multimodal sensors from an RGB camera, a 3D LiDAR, and an IMU, we adopt our previous work **Coco-LIC** [40], a continuous-time odometry to tightly fuse the sensor data, delivering robust, real-time, and accurate pose estimation even in challenging scenarios. It incrementally optimizes the trajectory for every 0.1 seconds via factor graph optimization, incorporating point-to-map LiDAR factors, frame-to-map visual factors, and inertial factors.

For photo-realistic reconstruction, 3D Gaussians in the map need to be bootstrapped by a set of 3D points. The better initialization leads to faster convergence and higher-quality rendering. Exactly, the LiDAR points provide handy and more precise geometric priors for Gaussian initialization [42]. Our system supports both solid-state and mechanical spinning LiDARs. Specifically, after finishing the

sliding-window optimization of the trajectory segment in the latest 0.1 seconds, we first downsample the LiDAR points in this time interval by retaining one out of every  $N_l$  points randomly. Then we colorize the remained LiDAR points by projection onto the latest image and remove those outside the corresponding camera FoV (field of view). Afterward, these colorized LiDAR points in the world frame are sent to the mapping thread, together with the posed latest image, regarded as a frame.

Though offering accurate geometric priors, the region scanned by the LiDAR is limited compared to the camera, especially for the repetitive mechanical spinning LiDAR. Thus, we propose maintaining a visual sliding window similar to **VINS-Mono** [50] to compute triangulated SFM points for compensation. The window consists of  $N_k$  keyframes carefully selected from the latest images based on co-visibility, and every keyframe contains detected Shi-Tomasi corners [51] tracked by the KLT sparse optical flow [52]. We triangulate a corner with the estimated keyframe poses and camera intrinsics once it is consecutively tracked in  $N_t$  keyframes. The newly triangulated SFM points will be combined with LiDAR points for supplementation.

### C. Photo-Realistic Mapping using Gaussian Splatting

The mapping module receives sequential frames from the tracking module to perform reconstruction. Each frame consists of an estimated camera pose, an undistorted image, and colored LiDAR points combined with visual SFM points.

1) *Initialization of 3D Gaussians*: The photo-realistic Gaussian map is bootstrapped with all 3D points of the first received frame. To be specific, for each point, we initialize a new Gaussian centered at its position with the zeroth degree of **SH** filled with its RGB color, opacity set to be  $o_a$ , and rotation set to be the identity matrix. To mitigate the aliasing artifacts [53], we assign smaller scales for the closer Gaussians, while larger scales for those far away from the image plane, namely  $\frac{d}{f} \mathbf{e}$ , where  $\mathbf{e}$  is a  $3 \times 1$  vector filled with 1, while  $d$  and  $f$  denote the depth of the point in the current camera frame and the camera focal length, respectively.

In unbounded outdoor scenes, improper modeling of the sky would enforce Gaussians on the object surfaces to incorrectly fit the appearance of the sky, undermining the

rendering quality. Thus we initialize a batch of sky Gaussians at random  $N_s$  positions on the upper hemisphere specified by the world coordinate system as the center and a large radius  $R$ . The zeroth degree of **SH** of each sky Gaussian is filled with white color and the opacity is set to be  $o_b$ . Also, the initial scales of the sky Gaussians are determined by the closest distances between them. Consequently, these background sky Gaussians fall behind the foreground Gaussians.

2) *Expansion of Gaussian Map*: Every frame after the first usually captures the geometry and appearance of the newly observed areas. However, LiDAR points from different frames may contain duplicate information. We take every fifth frame as a keyframe. When a keyframe is received, we first merge all points received in these five frames into a point cloud. To avoid redundancy, we then render **O** from the current keyframe image view according to Eq. (7) and generate a mask **M** to select pixels that are not reliable from the current Gaussian map and tend to observe new areas:

$$\mathbf{M} = \mathbf{O} < \tau. \quad (8)$$

Only points in the merged point cloud that can be projected onto the selected pixels are utilized to initialize new Gaussians as Sec. III-C.1 does, to expand the Gaussian map.

3) *Optimization*: Once the current received frame is a keyframe, we randomly select  $K$  keyframes from all keyframes to optimize the Gaussian map, avoiding the catastrophic forgetting problems and maintaining the geometric consistency of the global map. We randomly shuffle the selected  $K$  keyframes and iterate through each of them to optimize the map by minimizing re-rendering loss:

$$\mathcal{L} = (1 - \lambda) \|\mathbf{E}[\mathbf{C}] - \bar{\mathbf{C}}\|_1 + \lambda \mathcal{L}_{\text{D-SSIM}}, \quad (9)$$

where  $\bar{\mathbf{C}}$  and  $\mathbf{C}$  are the ground-truth image and the image rendered by Eq. (5), respectively.  $\mathcal{L}_{\text{D-SSIM}}$  is a D-SSIM term [2]. To handle the varying camera exposure, we additionally optimize a  $3 \times 4$  affine matrix **E** which is applied to the rendered color [54]. The first three columns of **E** are used for scaling, and the last column is used for offsetting.

#### D. Meticulous CUDA-related Designs for Acceleration

Implemented fully in C++ and CUDA, our SLAM system consists of a tracking module and a mapping module, running in parallel and exchanging data based on ROS. The mapping module is based on the LibTorch framework. Inspired by [55, 56], we carefully integrate and design a series of strategies related to CUDA to boost the training of Gaussians, an important aspect overlooked in previous works.

1) *Per-Gaussian Backpropagation*: During the backward pass of Gaussian Splatting [2], gradients are propagated from pixels to Gaussians, constituting the most computationally expensive part of training. Specifically, each pixel corresponds to a varying number of Gaussians, and the overall time cost is dictated by the pixel with the highest number of Gaussians. Consequently, all other CUDA threads remain idle, waiting for this bottleneck pixel to complete its backpropagation. Additionally, atomic gradient addition conflicts arise when multiple pixels backpropagate to the

same Gaussian. To mitigate this, we adopt a per-Gaussian backpropagation strategy [55], assigning CUDA threads to all Gaussians within a tile. This allows to enhance parallelism while reducing collisions.

2) *Tile Culling and Additional Tricks*: The vanilla 3DGS [2] approximates projected 2D splatted Gaussians as circles to determine relevant tiles. However, this approach is suboptimal for slim 2D Gaussians, which frequently appear in incremental mapping systems such as SLAM, as they contribute minimally to most tiles selected by a circular approximation. In our implementation, we efficiently cull these weakly affected tiles through load balancing [56], enhancing computational efficiency. For faster training of the Gaussian map representation, we further make use of the sparse Adam [55] to only update Gaussians in the current camera frustum instead of optimizing all the Gaussians.

## IV. EXPERIMENTS

### A. Experimental Setup

1) *Implementation Details*: Our evaluations are conducted on a desktop PC with an NVIDIA RTX 3090 GPU (24 GB VRAM), a 3.2GHz Intel Core i7-8700 CPU, and 32 GB of RAM. For mapping data preparation, we set  $N_l$  to 10,  $N_k$  to 11, and  $N_t$  to 9. For map initialization, we set  $o_a$  to 0.1,  $o_b$  to 0.7,  $N_s$  to  $1e5$ , and  $R$  to  $1e4$ . For map expansion, we set  $\tau$  to 0.99. As for map optimization, we set the loss weighting  $\lambda$  to 0.2 and the number  $K$  of selected keyframes to 100. All the learning rates for Gaussian attributes are kept the same with the vanilla [2] but not decayed with schedulers. We share the same learning rate of the affine matrix with [54]. Experiments across all tested scenarios use the same hyperparameters to ensure a fair and comprehensive evaluation.

2) *Datasets*: We conduct extensive experiments on three public LiDAR-Inertial-Camera datasets, including the FAST-LIVO dataset [38] and the R3LIVE dataset [37] with a solid-state LiDAR, and the MCD dataset [57] with a mechanical spinning LiDAR. We test 9 sequences among the three datasets, including (1) *hku2* (f0), *LiDAR\_Degenerate* (f1), and *Visual\_Challenge* (f2) from the FAST-LIVO dataset, (2) *hku\_campus\_seq\_00* (r0), *degenerate\_seq\_00* (r1), and *degenerate\_seq\_01* (r2) from the R3LIVE dataset, and (3) segments of *tuhh\_day\_02* (m0), *tuhh\_day\_03* (m1), and *tuhh\_day\_04* (m2) sequences in the large-scale MCD dataset [57]. Note that for all tested datasets, we only adopt the left images if the stereo images are provided. The FAST-LIVO dataset and the R3LIVE dataset provide  $640 \times 512$  images, while the MCD dataset has  $640 \times 480$  images.

3) *Baselines*: Note that when operating in unbounded outdoor environments, depth data from RGB-D cameras with a limited sensing range is often unreliable. Therefore, we first compare our method with the SOTA RGB-only systems, including the NeRF-based approach NeRF-SLAM [15] and the 3DGS-based method MonoGS [26]. Additionally, since *no open-source radiance-field-based LiDAR-Camera SLAM system is currently available*, we adapt the state-of-the-art RGB-D method SplaTAM [18] to operate with LiDAR and monocular images for comparison. Specifically, we merge

TABLE I: Quantitative rendering performance on public multimodal datasets. The best results are highlighted in bold. All the compared methods except ours utilize either the ground-truth poses or our estimated poses.

| Method            | Metric | f0           | f1           | f2           | r0           | r1           | r2           | m0           | m1           | m2           | Avg.         |
|-------------------|--------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| <i>Train View</i> |        |              |              |              |              |              |              |              |              |              |              |
| NeRF-SLAM [15]    | PSNR↑  | 25.56        | 25.47        | 17.01        | 19.53        | 21.20        | 15.02        | 18.70        | 18.93        | 15.40        | 19.65        |
|                   | SSIM ↑ | 0.629        | 0.702        | 0.513        | 0.645        | 0.534        | 0.711        | 0.632        | 0.635        | 0.428        | 0.603        |
|                   | LPIPS↓ | 0.281        | 0.272        | 0.512        | 0.455        | 0.364        | 0.328        | 0.449        | 0.447        | 0.535        | 0.405        |
| MonoGS [26]       | PSNR↑  | 23.58        | 23.45        | 17.04        | 16.19        | 15.03        | 15.09        | 16.41        | 13.69        | 14.91        | 17.27        |
|                   | SSIM ↑ | 0.609        | 0.762        | 0.671        | 0.543        | 0.490        | 0.512        | 0.457        | 0.502        | 0.384        | 0.548        |
|                   | LPIPS↓ | 0.643        | 0.757        | 0.643        | 0.714        | 0.670        | 0.710        | 0.755        | 0.767        | 0.734        | 0.710        |
| SplaTAM [18]      | PSNR↑  | 25.51        | 27.40        | 17.84        | 17.10        | 19.24        | 18.30        | 13.68        | 13.17        | 10.01        | 18.03        |
|                   | SSIM ↑ | 0.709        | 0.798        | 0.652        | 0.526        | 0.666        | 0.597        | 0.488        | 0.565        | 0.327        | 0.592        |
|                   | LPIPS↓ | 0.214        | 0.235        | 0.346        | 0.448        | 0.295        | 0.359        | 0.456        | 0.380        | 0.517        | 0.361        |
| Gaussian-LIC      | PSNR↑  | <b>29.89</b> | <b>31.28</b> | <b>23.90</b> | <b>25.27</b> | <b>22.47</b> | <b>23.49</b> | <b>21.06</b> | <b>22.87</b> | <b>20.73</b> | <b>24.55</b> |
|                   | SSIM ↑ | <b>0.820</b> | <b>0.840</b> | <b>0.830</b> | <b>0.805</b> | <b>0.794</b> | <b>0.811</b> | <b>0.693</b> | <b>0.740</b> | <b>0.595</b> | <b>0.770</b> |
|                   | LPIPS↓ | <b>0.155</b> | <b>0.178</b> | <b>0.173</b> | <b>0.212</b> | <b>0.192</b> | <b>0.187</b> | <b>0.330</b> | <b>0.266</b> | <b>0.383</b> | <b>0.231</b> |
| <i>Novel View</i> |        |              |              |              |              |              |              |              |              |              |              |
| Gaussian-LIC      | PSNR↑  | 29.28        | 30.91        | 23.28        | 24.52        | 22.03        | 22.59        | 20.19        | 22.12        | 19.57        | 23.83        |
|                   | SSIM ↑ | 0.799        | 0.830        | 0.811        | 0.785        | 0.776        | 0.777        | 0.639        | 0.693        | 0.531        | 0.738        |
|                   | LPIPS↓ | 0.160        | 0.180        | 0.179        | 0.215        | 0.196        | 0.195        | 0.336        | 0.272        | 0.391        | 0.236        |

point clouds from multiple LiDAR scans and obtain dense depth maps through projection, creating pseudo RGB-D images that can be used as input for SplaTAM. For a fair comparison, we adjust its opacity threshold  $\tau$  to match ours, allowing more Gaussians in unbounded scenes, which proves to be effective. To evaluate real-time performance, post-processing is disabled for all compared methods.

## B. Evaluation of Mapping

1) *Tracking*: Based on single-modality sensor input or simple re-rendering loss for tracking, all the compared methods get significant drift or even fail in pose estimation across all the tested sequences. On the contrary, our system consistently achieves superior tracking robustness and accuracy by tightly coupling LiDAR-Inertial-Camera data in a continuous-time factor graph [39].

2) *Rendering*: We analyze the performance of photo-realistic mapping by checking the quality of RGB rendering results. Metrics including Peak Signal to Noise Ratio (PSNR), Structural Similarity (SSIM), and Learned Perceptual Image Patch Similarity (LPIPS) are adopted for analysis [2]. Note that all compared methods struggle with pose estimation in complex scenarios. Therefore, we run them with *ground-truth poses* to purely evaluate their mapping performance. In the FAST-LIVO and R3LIVE datasets, where ground-truth poses are unavailable, we run the compared methods using the more accurate estimated poses from our approach instead. Table I shows the evaluation of rendering results on both the training views and novel views. Qualitative results are also shown in Fig. 2. Lack of precise priors for Gaussian initialization, MonoGS shows degraded performance outdoors. With additional depth supervision estimated from DroidSLAM [30], NeRF-SLAM achieves slightly improved results but at the cost of significantly increased rendering time. For improved running speed, SplaTAM represents the scene using isotropic Gaussians without considering view-dependent effects, sacrificing visual quality and tending to degenerate in complicated unbounded scenes. With superior geometric priors from a combination of LiDAR points and visual SFM points, and

TABLE II: Runtime (seconds) of different methods on the sequence f0 (duration: 105 seconds) with their estimated poses.

|                        | Tracking↓  | Mapping↓   | Total↓     | PSNR↑        |
|------------------------|------------|------------|------------|--------------|
| NeRF-SLAM [15]         | 105        | 288        | 288        | 25.23        |
| MonoGS [26]            | 181        | 387        | 387        | 21.42        |
| SplaTAM [18]           | 954        | 1620       | 2574       | 17.37        |
| COLMAP [58] + 3DGS [2] | -          | -          | 6764       | <b>32.56</b> |
| Gaussian-LIC           | <b>105</b> | <b>105</b> | <b>105</b> | 29.89        |
| Gaussian-LIC w/o acc   | 105        | 198        | 198        | 29.89        |

the appropriate modeling of sky and camera exposure, our system consistently achieves the best rendering quality across all sequences on the training views. Even more promisingly, the quality of our renderings at novel views surpasses that of other methods, even on their own training views.

3) *Runtime*: We evaluate the real-time performance of all compared methods, defining real-time as completing data processing within the duration of sensor data acquisition, without extensive post-processing. All approaches rely on their estimated poses rather than privileged ground-truth poses. For the sequential system SplaTAM, the total runtime is the sum of the time taken by both tracking and mapping modules, whereas for the parallel systems (NeRF-SLAM, MonoGS, Gaussian-LIC), the runtime is determined by the module with the longer time consumption. Tab. II presents the results on sequence f0 (105s duration). Powered by real-time LIC odometry and carefully designed strategies to accelerate Gaussian map optimization, our method is the only real-time-capable approach while also achieving the highest accuracy. It completes both tracking and mapping immediately after receiving all sensory data. We highlight the importance of the introduced acceleration strategies (see Sec. III-D) in Tab. II with the entry *Gaussian-LIC w/o acc*. Additionally, we show the results for the vanilla 3DGS [2] initialized by COLMAP [58], where the total runtime includes the offline SFM and Gaussian training. Although the offline, batch-based approach with globally consistent poses and extended optimization outperforms online incremental methods, it is unsuitable for real-time robotic applications.

Fig. 2: Qualitative rendering performance on the sequences f2, r0, m0 and m2.

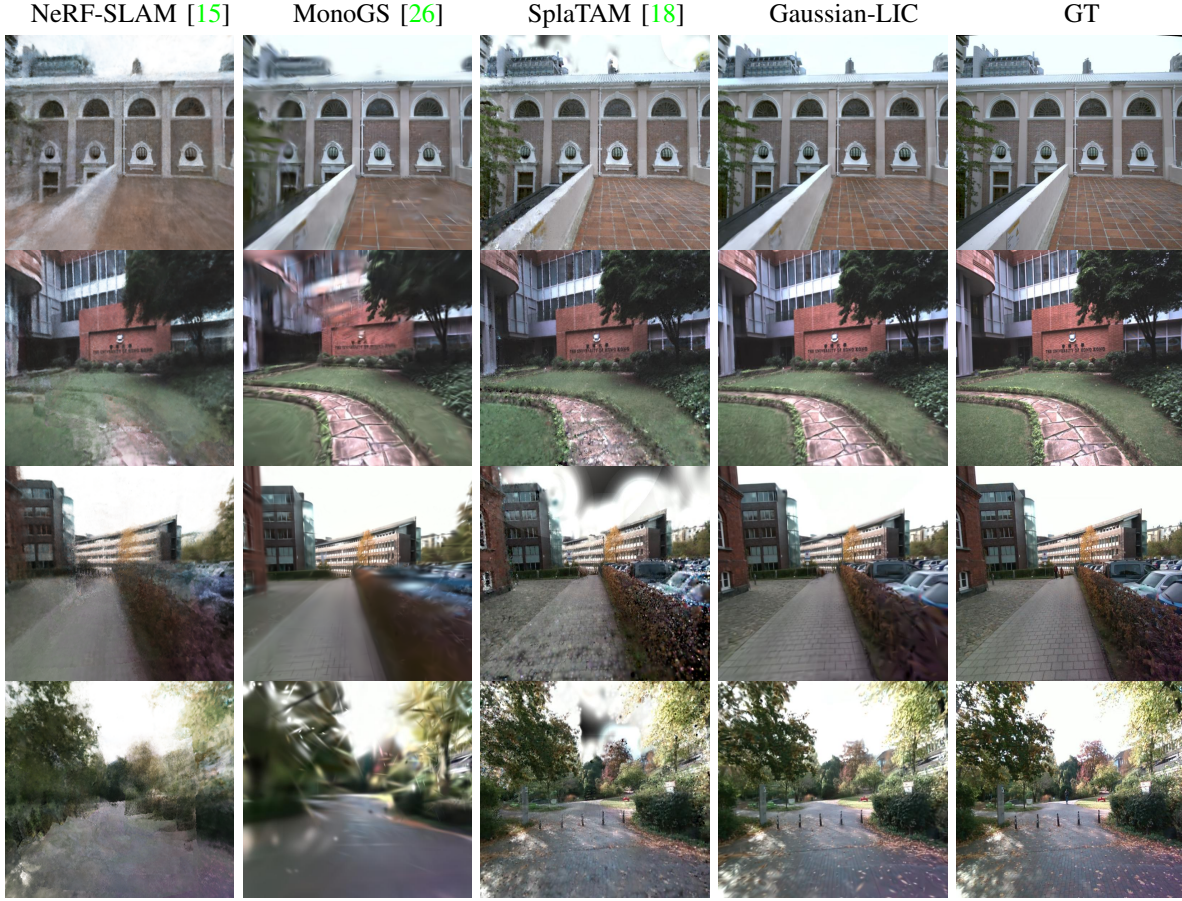


TABLE III: Ablation study on the sequence f0.

|                 | w/o expo. | w/o sky | w/o SFM | full         |
|-----------------|-----------|---------|---------|--------------|
| PSNR $\uparrow$ | 29.77     | 29.76   | 29.70   | <b>29.89</b> |

### C. Ablation Study

Table III presents the results of the ablation study on our proposed method. The removal of any individual component—such as modeling camera exposures, modeling the sky, or incorporating visual SFM points—leads to a decline in rendering quality. As illustrated in Fig. 3, the inclusion of sky Gaussians prevents surface Gaussians from incorrectly modeling the sky, thereby enhancing rendering fidelity. Additionally, integrating visual SFM points helps initialize 3D Gaussians in regions where LiDAR data is unavailable.

## V. CONCLUSIONS

In this study, we introduce a real-time, photo-realistic SLAM system that integrates LiDAR-Inertial-Camera odometry and Gaussian Splatting. Unlike previous radiance-field-based SLAM systems, which are primarily designed for indoor scenes with RGB-D or RGB sensors and struggle in unbounded outdoor scenes or under challenging conditions such as rapid motion and fluctuating lighting, our method is specifically developed to overcome these challenges outdoor. It enhances pose estimation and mapping quality by additionally fusing LiDAR and IMU data. Our SLAM system accommodates both solid-state and mechanical spinning



(a) w/o sky modeling (b) w/ sky modeling (c) SFM points

Fig. 3: The significance of sky modeling and incorporating SFM points. (a) and (b) are rendered images on the sequence r0. Without handling the sky, blurring occurs around the buildings (top left) because the surface Gaussians are trying to incorrectly fit the sky. (c) LiDAR points (colored red) cannot cover the entire scene in the camera FoV. We alleviate this problem by additionally incorporating visual SFM points (colored green) obtained by online triangulation.

LiDAR and utilizes visual SFM points to initialize Gaussians in areas not covered by the LiDAR. We’ve also incorporated sky modeling and camera exposure modeling for enhanced rendering. Our system also benefits from optimized strategies for fast Gaussian map optimization. Experiments show its superior performance. In the future, we will try to enhance the accuracy of the odometry by the reconstructed Gaussian map and improve the geometrical reconstruction quality.

## VI. ACKNOWLEDGEMENT

This work is supported by the “Leading Goose” Key R&D Program of Zhejiang Province in China (2025C01069).

## REFERENCES

- [1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. "Nerf: Representing scenes as neural radiance fields for view synthesis". In: *Communications of the ACM* 65.1 (2021), pp. 99–106.
- [2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. "3D Gaussian Splatting for Real-Time Radiance Field Rendering". In: *ACM Transactions on Graphics* 42.4 (2023).
- [3] F. Tosi, Y. Zhang, Z. Gong, E. Sandström, S. Mattoccia, M. R. Oswald, and M. Poggi. "How NeRFs and 3D Gaussian Splatting are Reshaping SLAM: a Survey". In: *arXiv preprint arXiv:2402.13255* (2024).
- [4] Y. Zheng, X. Chen, Y. Zheng, S. Gu, R. Yang, B. Jin, P. Li, C. Zhong, Z. Wang, L. Liu, et al. "GaussianGrasper: 3D Language Gaussian Splatting for Open-vocabulary Robotic Grasping". In: *arXiv preprint arXiv:2403.09637* (2024).
- [5] R. Jin, Y. Gao, H. Lu, and F. Gao. "GS-Planner: A Gaussian-Splatting-based Planning Framework for Active High-Fidelity Reconstruction". In: *arXiv preprint arXiv:2405.10142* (2024).
- [6] T. Chen, O. Shorinwa, W. Zeng, J. Bruno, P. Dames, and M. Schwager. "Splat-Nav: Safe Real-Time Robot Navigation in Gaussian Splatting Maps". In: *arXiv preprint arXiv:2403.02751* (2024).
- [7] X. Zuo, P. Samangouei, Y. Zhou, Y. Di, and M. Li. "Fmgs: Foundation model embedded 3d gaussian splatting for holistic 3d scene understanding". In: *International Journal of Computer Vision* (2024), pp. 1–17.
- [8] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison. "iMAP: Implicit mapping and positioning in real-time". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2021, pp. 6229–6238.
- [9] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys. "Nice-slam: Neural implicit scalable encoding for slam". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 12786–12796.
- [10] X. Yang, H. Li, H. Zhai, Y. Ming, Y. Liu, and G. Zhang. "Vox-Fusion: Dense tracking and mapping with voxel-based neural implicit representation". In: *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE. 2022, pp. 499–507.
- [11] H. Wang, J. Wang, and L. Agapito. "Co-SLAM: Joint Coordinate and Sparse Parametric Encodings for Neural Real-Time SLAM". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2023, pp. 13293–13302.
- [12] M. M. Johari, C. Carta, and F. Fleuret. "Eslam: Efficient dense slam system based on hybrid representation of signed distance fields". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2023, pp. 17408–17419.
- [13] E. Sandström, Y. Li, L. Van Gool, and M. R. Oswald. "Point-slam: Dense neural point cloud-based slam". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2023, pp. 18433–18444.
- [14] Z. Xin, Y. Yue, L. Zhang, and C. Wu. "HERO-SLAM: Hybrid Enhanced Robust Optimization of Neural SLAM". In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. 2024, pp. 8610–8616.
- [15] A. Rosinol, J. J. Leonard, and L. Carlone. "Nerf-slam: Real-time dense monocular slam with neural radiance fields". In: *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2023, pp. 3437–3444.
- [16] J. Naumann, B. Xu, S. Leutenegger, and X. Zuo. "NeRF-VO: Real-Time Sparse Visual Odometry with Neural Radiance Fields". In: *arXiv preprint arXiv:2312.13471* (2023).
- [17] C. Yan, D. Qu, D. Xu, B. Zhao, Z. Wang, D. Wang, and X. Li. "Gs-slam: Dense visual slam with 3d gaussian splatting". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 19595–19604.
- [18] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten. "SplatTAM: Splat Track & Map 3D Gaussians for Dense RGB-D SLAM". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 21357–21366.
- [19] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald. "Gaussian-slam: Photo-realistic dense slam with gaussian splatting". In: *arXiv preprint arXiv:2312.10070* (2023).
- [20] S. Ha, J. Yeon, and H. Yu. "Rgbd gs-icp slam". In: *arXiv preprint arXiv:2403.12550* (2024).
- [21] J. Hu, X. Chen, B. Feng, G. Li, L. Yang, H. Bao, G. Zhang, and Z. Cui. "CG-SLAM: Efficient Dense RGB-D SLAM in a Consistent Uncertainty-aware 3D Gaussian Field". In: *arXiv preprint arXiv:2403.16095* (2024).
- [22] Z. Peng, T. Shao, Y. Liu, J. Zhou, Y. Yang, J. Wang, and K. Zhou. "Rtg-slam: Real-time 3d reconstruction at scale using gaussian splatting". In: *ACM SIGGRAPH 2024 Conference Papers*. 2024, pp. 1–11.
- [23] F. A. Sarikamis and A. A. Alatan. "IG-SLAM: Instant Gaussian SLAM". In: *arXiv preprint arXiv:2408.01126* (2024).
- [24] P. Zhu, Y. Zhuang, B. Chen, L. Li, C. Wu, and Z. Liu. "MGS-SLAM: Monocular Sparse Tracking and Gaussian Mapping with Depth Smooth Regularization". In: *arXiv preprint arXiv:2405.06241* (2024).
- [25] H. Huang, L. Li, H. Cheng, and S.-K. Yeung. "Photo-SLAM: Real-time Simultaneous Localization and Photorealistic Mapping for Monocular Stereo and RGB-D Cameras". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 21584–21593.
- [26] H. Matsuki, R. Murai, P. H. Kelly, and A. J. Davison. "Gaussian splatting slam". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 18039–18048.
- [27] M. Li, S. Liu, and H. Zhou. "Sgs-slam: Semantic gaussian splatting for neural dense slam". In: *arXiv preprint arXiv:2402.03246* (2024).
- [28] S. Zhu, R. Qin, G. Wang, J. Liu, and H. Wang. "Semgauss-slam: Dense semantic gaussian splatting slam". In: *arXiv preprint arXiv:2403.07494* (2024).
- [29] Y. Ji, Y. Liu, G. Xie, B. Ma, Z. Xie, and H. Liu. "NEDS-SLAM: A Neural Explicit Dense Semantic SLAM Framework using 3D Gaussian Splatting". In: *IEEE Robotics and Automation Letters* (2024).
- [30] Z. Teed and J. Deng. "Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras". In: *Advances in neural information processing systems* 34 (2021), pp. 16558–16569.
- [31] T. Müller, A. Evans, C. Schied, and A. Keller. "Instant neural graphics primitives with a multiresolution hash encoding". In: *ACM Transactions on Graphics (ToG)* 41.4 (2022), pp. 1–15.
- [32] Z. Teed, L. Lipson, and J. Deng. "Deep patch visual odometry". In: *Advances in Neural Information Processing Systems* 36 (2024).
- [33] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós. "Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam". In: *IEEE Transactions on Robotics* 37.6 (2021), pp. 1874–1890.
- [34] X. Zuo, P. Geneva, W. Lee, Y. Liu, and G. Huang. "Lic-fusion: Lidar-inertial-camera odometry". In: *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2019, pp. 5848–5854.
- [35] X. Zuo, Y. Yang, P. Geneva, J. Lv, Y. Liu, G. Huang, and M. Pollefeys. "Lic-fusion 2.0: Lidar-inertial-camera odometry with sliding-window plane-feature tracking". In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2020, pp. 5112–5119.
- [36] T. Shan, B. Englot, C. Ratti, and D. Rus. "Lvi-sam: Tightly-coupled lidar-visual-inertial odometry via smoothing and mapping". In: *2021 IEEE international conference on robotics and automation (ICRA)*. IEEE. 2021, pp. 5692–5698.
- [37] J. Lin and F. Zhang. "R 3 LIVE: A Robust, Real-time, RGB-colored, LiDAR-Inertial-Visual tightly-coupled state Estimation and mapping package". In: *2022 International Conference on Robotics and Automation (ICRA)*. IEEE. 2022, pp. 10672–10678.
- [38] C. Zheng, Q. Zhu, W. Xu, X. Liu, Q. Guo, and F. Zhang. "FAST-LIVO: Fast and tightly-coupled sparse-direct LiDAR-inertial-visual odometry". In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2022, pp. 4003–4009.
- [39] J. Lv, X. Lang, J. Xu, M. Wang, Y. Liu, and X. Zuo. "Continuous-Time Fixed-Lag Smoothing for LiDAR-Inertial-Camera SLAM". In: *IEEE/ASME Transactions on Mechatronics* (2023).
- [40] X. Lang, C. Chen, K. Tang, Y. Ma, J. Lv, Y. Liu, and X. Zuo. "Coco-LIC: continuous-time tightly-coupled LiDAR-inertial-camera odometry using non-uniform B-spline". In: *IEEE Robotics and Automation Letters* (2023).
- [41] K. Rematas, A. Liu, P. P. Srinivasan, J. T. Barron, A. Tagliasacchi, T. Funkhouser, and V. Ferrari. "Urban radiance fields". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 12932–12942.
- [42] X. Zhou, Z. Lin, X. Shan, Y. Wang, D. Sun, and M.-H. Yang. "Driving-gaussian: Composite gaussian splatting for surrounding dynamic autonomous driving scenes". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 21634–21643.
- [43] Y. Yan, H. Lin, C. Zhou, W. Wang, H. Sun, K. Zhan, X. Lang, X. Zhou, and S. Peng. "Street Gaussians: Modeling Dynamic Urban Scenes with Gaussian Splatting". In: *ECCV*. 2024.
- [44] Y. Chen, C. Gu, J. Jiang, X. Zhu, and L. Zhang. "Periodic vibration gaussian: Dynamic urban scene reconstruction and real-time rendering". In: *arXiv preprint arXiv:2311.18561* (2023).
- [45] S. Hong, J. He, X. Zheng, C. Zheng, and S. Shen. "Liv-gaussmap: Lidar-inertial-visual fusion for real-time 3d radiance field map rendering". In: *IEEE Robotics and Automation Letters* (2024).
- [46] J. Cui, J. Cao, Y. Zhong, L. Wang, F. Zhao, P. Wang, Y. Chen, Z. He, L. Xu, Y. Shi, et al. "LetsGo: Large-Scale Garage Modeling and Rendering via LiDAR-Assisted Gaussian Primitives". In: *arXiv preprint arXiv:2404.09748* (2024).
- [47] C. Wu, Y. Duan, X. Zhang, Y. Sheng, J. Ji, and Y. Zhang. "MM-Gaussian: 3D Gaussian-based Multi-modal Fusion for Localization and Reconstruction in Unbounded Scenes". In: *arXiv preprint arXiv:2404.04026* (2024).
- [48] I. Vizzo, T. Guadagnino, B. Mersch, L. Wiesmann, J. Behley, and C. Stachniss. "Kiss-icp: In defense of point-to-point icp—simple, accurate, and robust registration if done the right way". In: *IEEE Robotics and Automation Letters* 8.2 (2023), pp. 1029–1036.
- [49] L. C. Sun, N. P. Bhatt, J. C. Liu, Z. Fan, Z. Wang, T. E. Humphreys, and U. Topcu. "MM3DGS SLAM: Multi-modal 3D Gaussian Splatting for SLAM Using Vision, Depth, and Inertial Measurements". In: *arXiv preprint arXiv:2404.00923* (2024).
- [50] T. Qin, P. Li, and S. Shen. "Vins-mono: A robust and versatile monocular visual-inertial state estimator". In: *IEEE transactions on robotics* 34.4 (2018), pp. 1004–1020.
- [51] J. Shi et al. "Good features to track". In: *1994 Proceedings of IEEE conference on computer vision and pattern recognition*. IEEE. 1994, pp. 593–600.

- [52] B. D. Lucas and T. Kanade. "An iterative image registration technique with an application to stereo vision". In: *IJCAI'81: 7th international joint conference on Artificial intelligence*. Vol. 2. 1981, pp. 674–679.
- [53] Z. Yan, W. F. Low, Y. Chen, and G. H. Lee. "Multi-scale 3d gaussian splatting for anti-aliased rendering". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 20923–20931.
- [54] B. Kerbl, A. Meuleman, G. Kopanas, M. Wimmer, A. Lanvin, and G. Drettakis. "A hierarchical 3d gaussian representation for real-time rendering of very large datasets". In: *ACM Transactions on Graphics (TOG)* 43.4 (2024), pp. 1–15.
- [55] S. S. Mallick, R. Goel, B. Kerbl, M. Steinberger, F. V. Carrasco, and F. De La Torre. "Taming 3dgs: High-quality radiance fields with limited resources". In: *SIGGRAPH Asia 2024 Conference Papers*. 2024, pp. 1–11.
- [56] L. Radl, M. Steiner, M. Parger, A. Weinrauch, B. Kerbl, and M. Steinberger. "Stopthepop: Sorted gaussian splatting for view-consistent real-time rendering". In: *ACM Transactions on Graphics (TOG)* 43.4 (2024), pp. 1–17.
- [57] T.-M. Nguyen, S. Yuan, T. H. Nguyen, P. Yin, H. Cao, L. Xie, M. Wozniak, P. Jensfelt, M. Thiel, J. Ziegenbein, and N. Blunder. "MCD: Diverse Large-Scale Multi-Campus Dataset for Robot Perception". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. June 2024.
- [58] J. L. Schönberger and J.-M. Frahm. "Structure-from-Motion Revisited". In: *Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016.